

# TD4 : Recherche récursive, division et requêtes complexes - Corrigé enseignant

## 03b4764

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : [www.mickael-martin-nevot.com](http://www.mickael-martin-nevot.com)

---

## 1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

### 1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne : <https://livesql.oracle.com>.

### 1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela consulter le document *Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier*. Ajouter ensuite une base de données nommée `zenetude-bd`.

## 2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D\_XXX<sup>1</sup>.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

---

1. Les domaines identiques sont surlignés avec la même couleur.

### 3 Base de données exemple

La base de données exemple, Questionnaire, utilisée dans les documents de travaux dirigés de cet enseignement, permet à un enseignant de base de données d'effectuer le suivi des étudiants lors des travaux pratiques de sa matière, en gérant les étudiants, les thèmes abordés, les questions et les évaluations.

#### 3.1 Dictionnaire de données

La base de données Questionnaire a été élaborée à partir du dictionnaire des données suivant <sup>2</sup> :

**Table 1** – Dictionnaire des données de la base de données exemple

| Attribut            | Description                               | Domaine                       | Remarques                     |
|---------------------|-------------------------------------------|-------------------------------|-------------------------------|
| numEt               | Numéro d'un étudiant                      | D_NUMET :<br>NUMBER(6,0)      | Valeurs uniques               |
| nom                 | Nom d'un étudiant                         | D_NOM :<br>VARCHAR2(20)       |                               |
| prenom              | Prénom d'un étudiant                      | D_PRENOM :<br>VARCHAR2(15)    |                               |
| typeBac             | Type du Bac d'un étudiant                 | D_TYPEBAC :<br>VARCHAR2(15)   | {GENERAL, ..., INTERNATIONAL} |
| groupe              | Numéro de groupe d'un étudiant            | D_GROUPE :<br>NUMBER(1,0)     | {1, 2, 3, 4}                  |
| idQ                 | Identifiant d'une question                | D_IDQ : NUMBER(6,0)           | Valeurs uniques               |
| numTP               | Numéro d'un TP                            | D_NUMTP :<br>NUMBER(1,0)      | {1, 2, 3, 4}                  |
| niveau              | Niveau de difficulté d'une question       | D_NIVEAU :<br>VARCHAR2(15)    | {FACILE, ..., COMPLEXE}       |
| temps (Question)    | Temps estimé pour répondre à une question | D_TEMPS :<br>NUMBER(2,0)      |                               |
| nbVariantes         | Nombre de variantes demandées             | D_NBVARIANTE :<br>NUMBER(1,0) | {1, 2, 3}                     |
| nbPoints (Question) | Nombres de points attribués à la question | D_NBPOINT :<br>NUMBER(2,0)    | {1, 2, 3, 4, 5}               |
| idT                 | Identifiant d'un thème                    | D_IDT : NUMBER(6,0)           | Valeurs uniques               |
| libelle             | Libellé d'un thème                        | D_LIBELLE :<br>VARCHAR2(40)   | Valeurs uniques               |
| idTPere             | Identifiant du thème père d'un thème      | D_IDT : NUMBER(6,0)           |                               |
| resultat            | Résultat d'une évaluation                 | D_RESULTAT :<br>VARCHAR2(30)  | {FAUX, ..., JUSTE}            |
| temps (Evaluer)     | Temps mis par un étudiant pour répondre   | D_TEMPS :<br>NUMBER(2,0)      |                               |
| nbVariantes         | Nombre de variantes réalisées             | D_NBVARIANTE :<br>NUMBER(1,0) |                               |
| nbPoints (Evaluer)  | Nombre de points obtenus                  | D_POINT :<br>NUMBER(4,2)      |                               |

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

### Remarques

Les thèmes sont organisés de manière hiérarchique. Quel que soit leur niveau dans la hiérarchie, tous les thèmes possèdent les mêmes attributs.

Voici la hiérarchie des thèmes de l'extension de la base de données exemple représentée sous forme d'arborescences des libellés :

### Remarques

Nous considérons qu'il n'y a pas deux étudiants homonymes ayant le même prénom et le même nom.

## 3.2 MCD

Le MCD (voir figure 2) modélise trois entités :

- **Etudiant** : les étudiants suivis lors des travaux pratiques
- **Question** : les questions posées dans les TP
- **Theme** : les thèmes abordés, organisés hiérarchiquement (association réflexive **A pour père**)

L'association **Traite** relie une question à un ou plusieurs thèmes. L'association **Evalue** relie un étudiant à une question et porte le résultat, le temps de réponse, le nombre de variantes réalisées et le nombre de points obtenus.

## 3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

**Etudiant** (numEt, nom, prenom, typeBac, groupe)

**Question** (idQ, numTP, niveau, temps, nbVariantes, nbPoints)

**Theme** (idT, libelle, *idTPere#*)

**Traite** (*idQ#*, *idT#*)

**Evalue** (*numEt#*, *idQ#*, resultat, temps, nbVariantes, nbPoints)

### Remarques

La clef étrangère *idTPere* dans la relation **Theme** fait référence à la clef primaire *idT*.

Les autres clefs étrangères font référence aux clefs primaires de même nom.

## 4 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

### 4.1 Expression des recherches récursives

**Q1** Donnez les identifiants et libellés des racines des hiérarchies des thèmes. *2 attributs, 2 tuples*

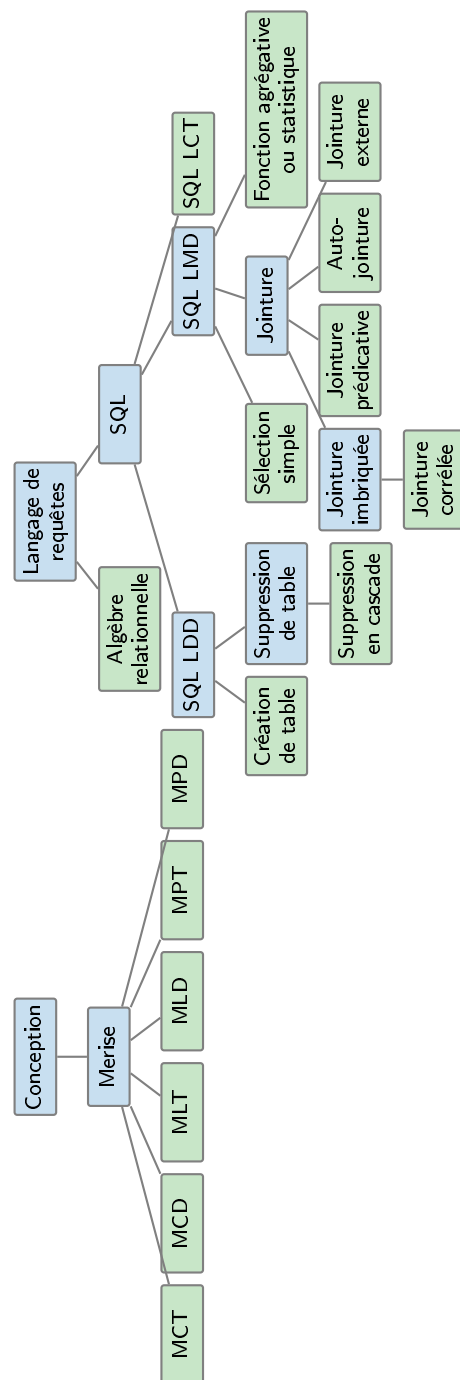


Figure 1 – Hiérarchie des thèmes



Figure 2 – Modèle conceptuel des données (MCD)

```

SELECT idT, libelle
FROM Theme
WHERE idTPere IS NULL;

```

**Q2** Donnez les identifiants et libellés des feuilles de la hiérarchie des thèmes. *2 attributs, 16 tuples*

```

SELECT idT, libelle
FROM Theme
WHERE idT NOT IN (
    SELECT idTPere
    FROM Theme
    WHERE idTPere IS NOT NULL
);

```

**Q3** Donnez les identifiants et libellés de la hiérarchie des fils du thème Langage de requêtes. *2 attributs, 17 tuples*

```

SELECT idT, libelle
FROM Theme
START WITH libelle = 'LANGAGE DE REQUETES'
CONNECT BY idTPere = PRIOR idT;

```

Version CTE récursive.

```

WITH RECURSIVE DescTheme (idT, libelle) AS (
    SELECT idT, libelle
    FROM Theme
    WHERE libelle = 'LANGAGE DE REQUETES'
    UNION ALL
    SELECT T.idT, T.libelle
    FROM Theme T

```

```

        INNER JOIN DescTheme D ON T.idTPere = D.idT
    )
    SELECT idT, libelle
    FROM DescTheme;

```

**Q4** Donnez les identifiants et libellés des thèmes subordonnés directement ou pas (l'ensemble des thèmes plus spécifiques) au thème Jointure à l'exception du sous-thème Jointure imbriquée. *2 attributs, 5 tuples*

```

SELECT idT, libelle
FROM Theme
WHERE libelle <> 'JOINTURE IMBRIQUEE'
START WITH libelle = 'JOINTURE'
CONNECT BY idTPere = PRIOR idT;

```

Version CTE récursive.

```

WITH RECURSIVE DescTheme (idT, libelle) AS (
    SELECT idT, libelle
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.libelle
    FROM Theme T
        INNER JOIN DescTheme D ON T.idTPere = D.idT
)
SELECT idT, libelle
FROM DescTheme
WHERE libelle <> 'JOINTURE IMBRIQUEE';

```

**Q5** Donnez les identifiants et libellés des thèmes subordonnés directement ou pas au thème Jointure à l'exception du thème Jointure imbriquée et de ses sous-thèmes. *2 attributs, 4 tuples*

```

SELECT idT, libelle
FROM Theme
START WITH libelle = 'JOINTURE'
CONNECT BY
    idTPere = PRIOR idT
    AND libelle <> 'JOINTURE IMBRIQUEE';

```

Version CTE récursive.

```

WITH RECURSIVE DescTheme (idT, libelle) AS (
    SELECT idT, libelle
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.libelle
    FROM Theme T
        INNER JOIN DescTheme D ON T.idTPere = D.idT
    WHERE T.libelle <> 'JOINTURE IMBRIQUEE'
)

```

```
)
SELECT idT, libelle
FROM DescTheme;
```

**Q6** Donnez les identifiants et libellés généralisant strictement le thème Jointure externe en triant du thème le plus général au thème le plus spécifique. *2 attributs, 4 tuples*

```
SELECT idT, libelle
FROM Theme
WHERE libelle <> 'JOINTURE EXTERNE'
START WITH libelle = 'JOINTURE EXTERNE'
CONNECT BY idT = PRIOR idTPere
ORDER BY LEVEL DESC;
```

Version CTE récursive.

```
WITH RECURSIVE AncTheme (idT, libelle, idTPere, lvl) AS (
    SELECT idT, libelle, idTPere, 1
    FROM Theme
    WHERE libelle = 'JOINTURE EXTERNE'
    UNION ALL
    SELECT T.idT, T.libelle, T.idTPere, A.lvl + 1
    FROM Theme T
    INNER JOIN AncTheme A ON T.idT = A.idTPere
)
SELECT idT, libelle
FROM AncTheme
WHERE libelle <> 'JOINTURE EXTERNE'
ORDER BY lvl DESC;
```

**Q7** Quels sont les identifiants des questions et numéros de TP se rapportant à un thème subordonné directement ou indirectement au thème Jointure ? *2 attributs, 28 tuples*

V1.

```
SELECT DISTINCT Q.idQ, Q.numTP
FROM Question Q
    INNER JOIN Traite TQ
        ON
            Q.idQ = TQ.idQ
            AND idT IN (
                SELECT idT
                FROM Theme
                START WITH libelle = 'JOINTURE'
                CONNECT BY idTPere = PRIOR idT
            );
```

V2.

```
SELECT idQ, numTP
FROM Question
```

```

WHERE idQ IN (
    SELECT idQ
    FROM Traite
    WHERE idT IN (
        SELECT idT
        FROM Theme
        START WITH libelle = 'JOINTURE'
        CONNECT BY idTPere = PRIOR idT
    )
);

```

Version CTE récursive.

```

WITH RECURSIVE DescTheme (idT) AS (
    SELECT idT
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT
    FROM Theme T
    INNER JOIN DescTheme D ON T.idTPere = D.idT
)
SELECT DISTINCT Q.idQ, Q.numTP
FROM Question Q
    INNER JOIN Traite TQ
        ON Q.idQ = TQ.idQ
WHERE TQ.idT IN (SELECT idT FROM DescTheme);

```

**Q8** Quels sont les identifiants et libellés des thèmes généralisant les thèmes Jointure ou Sélection simple? *2 attributs, 5 tuples*

**V1.**

```

SELECT DISTINCT idT, libelle
FROM Theme
START WITH libelle IN ('JOINTURE', 'SELECTION SIMPLE')
CONNECT BY idT = PRIOR idTPere;

```

**V2.**

```

SELECT idT, libelle
FROM Theme
START WITH libelle IN ('JOINTURE')
CONNECT BY idT = PRIOR idTPere
UNION
SELECT idT, libelle
FROM Theme
START WITH libelle IN ('SELECTION SIMPLE')
CONNECT BY idT = PRIOR idTPere;

```

Version CTE récursive.



```
WITH RECURSIVE AncTheme (idT, libelle, idTPere) AS (
    SELECT idT, libelle, idTPere
    FROM Theme
    WHERE libelle IN ('JOINTURE', 'SELECTION SIMPLE')
    UNION ALL
    SELECT T.idT, T.libelle, T.idTPere
    FROM Theme T
        INNER JOIN AncTheme A ON T.idT = A.idTPere
)
SELECT DISTINCT idT, libelle
FROM AncTheme;
```

**Q9** Quels sont les identifiants et libellés des thèmes généralisant les thèmes Jointure et Sélection simple? *2 attributs, 3 tuples*

**V1.**

```
SELECT idT, libelle
FROM Theme
START WITH libelle = 'JOINTURE'
CONNECT BY idT = PRIOR idTPere
INTERSECT
SELECT idT, libelle
FROM Theme
START WITH libelle = 'SELECTION SIMPLE'
CONNECT BY idT = PRIOR idTPere;
```

**V2.**

```
SELECT idT, libelle
FROM Theme
WHERE idT IN (
    SELECT idT
    FROM Theme
    START WITH libelle = 'SELECTION SIMPLE'
    CONNECT BY idT = PRIOR idTPere
)
START WITH libelle = 'JOINTURE'
CONNECT BY idT = PRIOR idTPere;
```

**Version CTE récursive.**

```
WITH RECURSIVE AncJointure (idT, libelle, idTPere) AS (
    SELECT idT, libelle, idTPere
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.libelle, T.idTPere
    FROM Theme T
        INNER JOIN AncJointure A ON T.idT = A.idTPere
),
```

```
AncSelection (idT, libelle, idTPere) AS (
    SELECT idT, libelle, idTPere
    FROM Theme
    WHERE libelle = 'SELECTION SIMPLE'
    UNION ALL
    SELECT T.idT, T.libelle, T.idTPere
    FROM Theme T
        INNER JOIN AncSelection A ON T.idT = A.idTPere
)
SELECT idT, libelle
FROM AncJointure
INTERSECT
SELECT idT, libelle
FROM AncSelection;
```

**Q10** Quels sont les identifiants et libellés des thèmes subordonnés directement ou indirectement au thème SQL LDD et qui ne sont utilisés par aucune question ? *2 attributs, 3 tuples*

**V1.**

```
SELECT idT, libelle
FROM Theme
WHERE idT NOT IN (
    SELECT idT
    FROM Traite
)
START WITH libelle = 'SQL LDD'
CONNECT BY PRIOR idT = idTPere;
```

**V2.**

```
SELECT idT, libelle
FROM Theme
START WITH libelle = 'SQL LDD'
CONNECT BY PRIOR idT = idTPere
EXCEPT
SELECT T.idT, libelle
FROM Theme T
    INNER JOIN Traite TQ
        ON T.idT = TQ.idT;
```

**V3.**

```
SELECT idT, libelle
FROM Theme
WHERE idT IN (
    SELECT idT
    FROM Theme
    START WITH libelle = 'SQL LDD'
    CONNECT BY PRIOR idT = idTPere
    EXCEPT
```

```

SELECT idT
FROM Traite
);

```

Version CTE récursive.

```

WITH RECURSIVE DescTheme (idT, libelle) AS (
    SELECT idT, libelle
    FROM Theme
    WHERE libelle = 'SQL LDD'
    UNION ALL
    SELECT T.idT, T.libelle
    FROM Theme T
        INNER JOIN DescTheme D ON T.idTPere = D.idT
)
SELECT idT, libelle
FROM DescTheme
WHERE idT NOT IN (
    SELECT idT
    FROM Traite
);

```

**Q11** Quels sont les identifiants et libellés des thèmes feuilles de la hiérarchie des thèmes qui sont subordonnés directement ou indirectement à la même racine que celle du thème Jointure.  
*2 attributs, 10 tuples*

```

SELECT idT, libelle
FROM Theme
WHERE idT NOT IN (
    SELECT idTPere
    FROM Theme
    WHERE idTPere IS NOT NULL
)
START WITH idT = (
    SELECT idT
    FROM Theme
    WHERE idTPere IS NULL
    CONNECT BY idT = PRIOR idTPere
    START WITH libelle = 'JOINTURE'
)
CONNECT BY PRIOR idT = idTPere;

```

Version CTE récursive.

```

WITH RECURSIVE AncJointure (idT, idTPere) AS (
    SELECT idT, idTPere
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.idTPere
    FROM Theme T

```

```

        INNER JOIN AncJointure A ON T.idT = A.idTPere
    ),
    RacineJointure AS (
        SELECT idT FROM AncJointure WHERE idTPere IS NULL
    ),
    DescRacine (idT, libelle) AS (
        SELECT idT, libelle
        FROM Theme
        WHERE idT = (SELECT idT FROM RacineJointure)
        UNION ALL
        SELECT T.idT, T.libelle
        FROM Theme T
        INNER JOIN DescRacine D ON T.idTPere = D.idT
    )
    SELECT idT, libelle
    FROM DescRacine
    WHERE idT NOT IN (
        SELECT idTPere
        FROM Theme
        WHERE idTPere IS NOT NULL
    );

```

**Q12** Quels sont les identifiants et libellés des thèmes se situant dans la branche, au-dessus ou en dessous, du thème Jointure ? *2 attributs, 9 tuples*

```

SELECT idT, libelle
FROM Theme
START WITH libelle = 'JOINTURE'
CONNECT BY idT = PRIOR idTPere
UNION
SELECT idT, libelle
FROM Theme
START WITH libelle = 'JOINTURE'
CONNECT BY PRIOR idT = idTPere;

```

Version CTE récursive.

```

WITH RECURSIVE AncJointure (idT, libelle, idTPere) AS (
    SELECT idT, libelle, idTPere
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.libelle, T.idTPere
    FROM Theme T
    INNER JOIN AncJointure A ON T.idT = A.idTPere
),
DescJointure (idT, libelle) AS (
    SELECT idT, libelle
    FROM Theme
    WHERE libelle = 'JOINTURE'
    UNION ALL
    SELECT T.idT, T.libelle

```

```

    FROM Theme T
      INNER JOIN DescJointure D ON T.idTPere = D.idT
  )
  SELECT idT, libelle FROM AncJointure
  UNION
  SELECT idT, libelle FROM DescJointure;

```

## 4.2 Expression des divisions

**Q13** Donnez les numéros, noms et prénoms des étudiants ayant fait toutes les questions du TP numéro 1. Proposer deux formulations différentes, avec double NOT EXISTS et HAVING, de cette requête. *3 attributs, 4 tuples*

Version double NOT EXISTS.

```

SELECT numEt, nom, prenom
FROM Etudiant Et
WHERE NOT EXISTS (
  SELECT *
  FROM Question Q
  WHERE
    numTP = 1
    AND NOT EXISTS (
      SELECT *
      FROM Evalue E
      WHERE
        Et.numEt = E.numEt
        AND Q.idQ = E.idQ
    )
);

```

### Remarques

Cette requête correspond au paraphrasage “Quels sont les numéros, noms et prénoms des étudiants ayant fait autant de questions du TP numéro 1 qu’il en existe?”

Version HAVING.

```

SELECT Et.numEt, nom, prenom
FROM Etudiant Et
  INNER JOIN Evalue E
    ON Et.numEt = E.numEt
  INNER JOIN Question Q
    ON E.idQ = Q.idQ
WHERE numTP = 1
GROUP BY Et.numEt, nom, prenom
HAVING COUNT(*) = (
  SELECT COUNT(*)
  FROM Question
  WHERE numTP = 1
);

```

**Remarques**

Cette requête correspond au paraphrasage “Quels sont les numéros, noms et prénoms des étudiants tels qu’il n’existe aucune question du TP numéro 1 qu’ils n’aient pas faite?”

**Q14** Quels sont les groupes d’étudiants dans lesquels tous les types de BAC sont représentés ? Proposer deux formulations différentes, avec double NOT EXISTS et HAVING, de cette requête.

*1 attribut, 3 tuples*

**Version double NOT EXISTS.**

```
SELECT DISTINCT groupe
FROM Etudiant Et
WHERE NOT EXISTS (
    SELECT *
    FROM Etudiant Et2
    WHERE NOT EXISTS (
        SELECT *
        FROM Etudiant Et3
        WHERE
            Et.groupe = Et3.groupe
            AND Et2.typeBAC = Et3.typeBAC
    )
);
```

**Remarques**

Cette requête correspond au paraphrasage “Quels sont les groupes d’étudiants pour lesquels il n’existe pas de type de BAC qui ne soit pas représenté ?

**Version HAVING.**

```
SELECT groupe
FROM Etudiant
GROUP BY groupe
HAVING COUNT(DISTINCT typeBAC) = (
    SELECT COUNT(DISTINCT typeBAC)
    FROM Etudiant
);
```

**Remarques**

Cette requête correspond au paraphrasage “Quels sont les groupes d’étudiants ayant autant de Types de BAC qu’il en existe?”

**Q15** Quels sont les groupes d’étudiants pour lesquels au moins un étudiant a été évalué pour chaque TP ? *1 attribut, 2 tuples*

**Version double NOT EXISTS.**

```
SELECT DISTINCT groupe
FROM Etudiant Et
```

```

WHERE NOT EXISTS (
    SELECT *
    FROM Question Q
    WHERE NOT EXISTS (
        SELECT *
        FROM Evaluate E
            INNER JOIN Etudiant Et2
                ON E.numEt = Et2.numEt
            INNER JOIN Question Q2
                ON E.idQ = Q2.idQ
        WHERE
            Et.groupe = Et2.groupe
            AND Q.numTP = Q2.numTP
    )
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quels sont les groupes pour lesquels il y a autant de TP évalués qu’il en existe?”

#### Version HAVING.

```

SELECT groupe
FROM Etudiant Et
    INNER JOIN Evaluate E
        ON Et.numEt = E.numEt
    INNER JOIN Question Q
        ON E.idQ = Q.idQ
GROUP BY groupe
HAVING COUNT(DISTINCT numTP) = (
    SELECT COUNT(DISTINCT numTP)
    FROM Question
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quels sont les groupes tels qu’il n’existe aucun TP pour lequel ils n’aient pas été évalué?”

### 4.3 Requetes complexes

**Q16** Donnez les numéros, noms et prénoms des étudiants ayant eu au moins un résultat faux au TP numéro 2 et au TP numéro 3. Proposer trois formulations différentes. *3 attributs, 1 tuple*

V1.

```

SELECT DISTINCT Et.numEt, nom, prenom
FROM Etudiant Et
    INNER JOIN Evaluate E
        ON Et.numEt = E.numEt
    INNER JOIN Question Q

```

```

        ON E.idQ = Q.idQ
    INNER JOIN Evaluate E2
        ON Et.numEt = E2.numEt
    INNER JOIN Question Q2
        ON E2.idQ = Q2.idQ
WHERE
    E.resultat = 'FAUX'
    AND Q.numTP = 2
    AND E2.resultat = 'FAUX'
    AND Q2.numTP = 3;

```

V2.

```

SELECT DISTINCT Et.numEt, nom, prenom
FROM Etudiant Et
    INNER JOIN Evaluate E
        ON Et.numEt = E.numEt
    INNER JOIN Question Q
        ON E.idQ = Q.idQ
WHERE
    resultat = 'FAUX'
    AND numTP = 2
    AND Et.numEt IN (
        SELECT Et.numEt
        FROM Etudiant Et
            INNER JOIN Evaluate E
                ON Et.numEt = E.numEt
            INNER JOIN Question Q
                ON
                    E.idQ = Q.idQ
                    AND E.idQ = Q.idQ
                    AND resultat = 'FAUX'
                    AND numTP = 3
    );

```

V3.

```

SELECT numEt, nom, prenom
FROM Etudiant
WHERE numEt IN (
    SELECT numEt
    FROM Evaluate E
        INNER JOIN Question Q
            ON E.idQ = Q.idQ
    WHERE
        resultat = 'FAUX'
        AND numTP = 2
    INTERSECT
    SELECT numEt
    FROM Evaluate E
        INNER JOIN Question Q

```



```

        ON E.idQ = Q.idQ
WHERE
    resultat = 'FAUX'
    AND numTP = 3
);

```

V4.

```

SELECT numEt, nom, prenom
FROM Etudiant
WHERE numEt IN (
    SELECT numEt
    FROM Evalue E
        INNER JOIN Question Q
            ON E.idQ = Q.idQ
    WHERE
        resultat = 'FAUX'
        AND numTP = 2
        AND numEt IN (
            SELECT numEt
            FROM Evalue E
                INNER JOIN Question Q
                    ON E.idQ = Q.idQ
            WHERE
                resultat = 'FAUX'
                AND numTP = 3
        )
);

```

**Q17** Donnez les numéros des groupes d'étudiants dont aucun étudiant n'a obtenu un résultat faux au TP numéro 1. *1 attribut, 1 tuple (4)*

```

SELECT DISTINCT groupe
FROM Etudiant
WHERE groupe NOT IN (
    SELECT groupe
    FROM Etudiant Et
        INNER JOIN Evalue E
            ON Et.numEt = E.numEt
        INNER JOIN Question Q
            ON E.idQ = Q.idQ
    WHERE
        resultat = 'FAUX'
        AND numTP = 1
);

```

**Q18** Donnez les numéros, noms et prénoms des étudiants dans le même groupe et ayant le même type de Bac que l'étudiante Marie Dujardin. *3 attributs, 7 tuples*

V1.

```

SELECT Et.numEt, Et.nom, Et.prenom
FROM Etudiant Et

```

```

INNER JOIN Etudiant EtMD
  ON
    Et.groupe = EtMD.groupe
  AND Et.typeBAC = EtMD.typeBAC
WHERE
  EtMD.nom = 'DUJARDIN'
  AND EtMD.prenom = 'MARIE';

```

V2.

```

SELECT numEt, nom, prenom
FROM Etudiant
WHERE (groupe, typeBAC) IN (
  SELECT groupe, typeBAC
  FROM Etudiant
  WHERE nom = 'DUJARDIN'
  AND prenom = 'MARIE'
);

```

**Q19** Donnez les numéros, noms et prénoms des étudiants ayant réalisés le nombre de variantes demandées pour au moins une Question du TP numéro 2. *3 attributs, 19 tuples*

V1.

```

SELECT DISTINCT Et.numEt, Et.nom, Et.prenom
FROM Etudiant Et
  INNER JOIN Evaluer Ev
    ON Et.numEt = Ev.numEt
  INNER JOIN Question Q
    ON
      Ev.idQ = Q.idQ
      AND Ev.nbVariantes = Q.nbVariantes
WHERE numTP = 2;

```

V2.

```

SELECT DISTINCT Et.numEt, Et.nom, Et.prenom
FROM Etudiant Et
  INNER JOIN Evaluer E ON Et.numEt = E.numEt
WHERE (idQ, nbVariantes) IN (
  SELECT idQ, nbVariantes
  FROM Question
  WHERE numTP = 2
);

```

**Q20** Donnez l'identifiant des questions et les numéros de TP portant à la fois sur le thème Jointure imbriquée et sur le thème Fonction agrégative ou statistique. *2 attributs, 2 tuples*

V1.

```

SELECT Q.idQ, Q.numTP
FROM Question Q

```

```

INNER JOIN Traite TQ
  ON Q.idQ = TQ.idQ
INNER JOIN Theme T
  ON TQ.idT = T.idT
WHERE
  T.libelle = 'JOINTURE IMBRIQUEE'
  AND Q.idQ IN (
    SELECT Q.idQ
    FROM Question Q
      INNER JOIN Traite TQ
        ON Q.idQ = TQ.idQ
      INNER JOIN Theme T
        ON TQ.idT = T.idT
    WHERE libelle = 'FONCTION AGREGATIVE OU STATISTIQUE'
  );

```

V2.

```

SELECT Q.idQ, Q.numTP
FROM Question Q
  INNER JOIN Traite TQ
    ON Q.idQ = TQ.idQ
  INNER JOIN Theme T
    ON TQ.idT = T.idT
WHERE T.libelle = 'JOINTURE IMBRIQUEE'
INTERSECT
SELECT Q.idQ, Q.numTP
FROM Question Q
  INNER JOIN Traite TQ
    ON Q.idQ = TQ.idQ
  INNER JOIN Theme T
    ON TQ.idT = T.idT
WHERE T.libelle = 'FONCTION AGREGATIVE OU STATISTIQUE';

```

**Q21** Donnez, pour chaque TP, le nombre de points obtenu par l'étudiante Marie Dujardin. 2 *attributs, 4 tuples*

```

SELECT numTP, SUM(E.NbPoints) AS totalPoints
FROM Etudiant Et
  INNER JOIN Evaluer E
    ON Et.numEt = E.numEt
  INNER JOIN Question Q
    ON E.idQ = Q.idQ
WHERE
  nom = 'DUJARDIN'
  AND prenom = 'MARIE'
GROUP BY numTP;

```

**Q22** Donnez les numéros et noms des étudiants ayant eu au moins trois résultats justes à des questions complexes. 2 *attributs, 3 tuples*

```

SELECT Et.numEt, Et.nom
FROM Etudiant Et

```

```
INNER JOIN Evaluate Ev
  ON Et.numEt = Ev.numEt
INNER JOIN Question Q
  ON Ev.idQ = Q.idQ
WHERE
  Niveau = 'COMPLEXE'
  AND resultat = 'JUSTE'
GROUP BY Et.numEt, nom
HAVING COUNT(*) >= 3;
```

**Q23** Donnez, pour chaque étudiant ayant au moins une réponse juste au TP numéro 3, le nombre de réponses effectivement justes à ce TP. *3 attributs, 12 tuples*

```
SELECT Et.numEt, Et.nom, COUNT(*) AS nbJustes
FROM Etudiant Et
  INNER JOIN Evaluate E
    ON Et.numEt = E.numEt
WHERE (Et.numEt, idQ) IN (
  SELECT numEt, E.idQ
  FROM Evaluate E
    INNER JOIN Question Q
      ON
        E.idQ = Q.idQ
        AND E.idQ = Q.idQ
  WHERE
    resultat = 'JUSTE'
    AND numTP = 3
)
GROUP BY Et.numEt, nom;
```